Studio Nx

Backend(FastAPI/Python) 24 — GET /api/v1/auth/check-nickname

2026-10-12

Backend(FastAPI/Python) 24 — 라우터 `GET /api/v1/auth/check-nickname`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `check_nickname`)
작업	P1-04 회원가입: 닉네임 실시간 중복 확인 API (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	23 `GET /auth/check-email` · 25 입력 스키마

1. 하는 일

주소의 `?nickname=` 값을 받아 **길이를 검사**하고, 20번 `check_nickname()`의 결과를 그대로 돌려줍니다. 23번과 같은 모양입니다.

요청	응답
`GET /api/v1/auth/check-nickname?nickname=fresh_nick`	200 `{"available": true, …}`
`…?nickname=ROUTE_USER` (`route_user`가 이미 있음)	200 `{"available": false, "reason": "TAKEN", …}`
`…?nickname=a` (1자), 21자, 빈 문자열	**422**이고, `details.nickname`에 오류가 담깁니다

2. 언제 쓰나

화면	버튼
join.html (P1-07)	닉네임 칸 옆의 **Check**
userInfo.html (P1-10)	닉네임을 바꾸기 전에 확인할 때 같은 API를 씁니다

응답에 따른 문구는 23번과 같습니다 (✓ Available / This nickname is already in use. / You can use it again after …).

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    NICKNAME_MIN,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
```

```
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`NICKNAME_MIN`을 이번에 추가했습니다.** 이 함수는 `Annotated`, `Query`, `Db`, `NICKNAME_MIN`, `NICKNAME_MAX`, `AvailabilityResponse`, `auth_service`를 사용합니다.

함수

```
@router.get("/check-nickname", response_model=AvailabilityResponse)
def check_nickname(
    db: Db,
    nickname: Annotated[str, Query(min_length=NICKNAME_MIN, max_length=NICKNAME_MAX)],
) -> AvailabilityResponse:
    return auth_service.check_nickname(db, nickname)
```

템플릿에는 `Query(max_length=NICKNAME_MAX)`만 있었습니다. **`min_length=NICKNAME_MIN`을 추가**하고 몸통 한 줄을 채웠습니다.

문법 풀이

import 문법	뜻
`from app.schemas.auth import (…, NICKNAME_MIN, …)`	괄호 안에 한 줄에 하나씩 적은 여러 이름을 가져옵니다

식	뜻
`Query(min_length=2, max_length=20)`	주소의 `?nickname=`에서 읽고, 2~20자여야 합니다. 틀리면 라우터를 실행하기 전에 422로 응답합니다 (02번 처리기)
`auth_service.check_nickname(db, nickname)`	20번을 `release=False`(기본값)로 부릅니다. 그래서 DB를 바꾸지 않습니다

4. 흐름

1. FastAPI가 `?nickname=`을 읽어 길이를 검사합니다. 틀리면 422로 응답합니다.
2. 20번으로 쓸 수 있는 닉네임인지 확인합니다 (앞뒤 공백을 지우고, 대소문자를 무시하며, DB는 바꾸지 않습니다).
3. 결과를 200으로 돌려줍니다.

5. 왜 이렇게

선택	이유
`min_length`를 추가함	템플릿대로라면 `"a"`(1자)를 물었을 때 "✓ Available"이 나오지만, 가입(22)은 2자 미만이라서 422로 막습니다. 확인과 가입의 규칙이 같아야 "쓸 수 있다더니 가입이 안 된다"는 일이 생기지 않습니다
나머지는 23번과 같음	확인만 하고(`GET`), "쓸 수 없음"도 200으로 응답하며, 형식이나 길이 오류만 422로 응답합니다

남은 차이 (25번에서 고칩니다): 지금은 앞뒤 공백을 **지우기 전**의 글자 수로 길이를 잽니다. 그래서 `" a "`는 5자로 통과한 뒤 `"a"`로 확인됩니다. 25번에서 "공백을 먼저 지우고 길이를 재는" 닉네임 타입을 만들어서, 가입 스키마와 이 라우터가 함께 쓰게 합니다.

6. 시험 (`tests/test_auth_routes.py`에 8개를 추가해서 모두 24개, 실제 앱과 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 닉네임	200, `available: true`
가입된 `route_user`를 3가지 방식으로 물음	`route_user`, `ROUTE_USER`, `" route_user "`가 모두 `TAKEN`입니다
길이가 틀린 3개	1자, 21자, 빈 문자열은 422이고, `details`에 `nickname`이 담깁니다
`nickname`이 없음	422

전체 시험 167개가 통과했고, `ruff` 검사도 통과했습니다.